

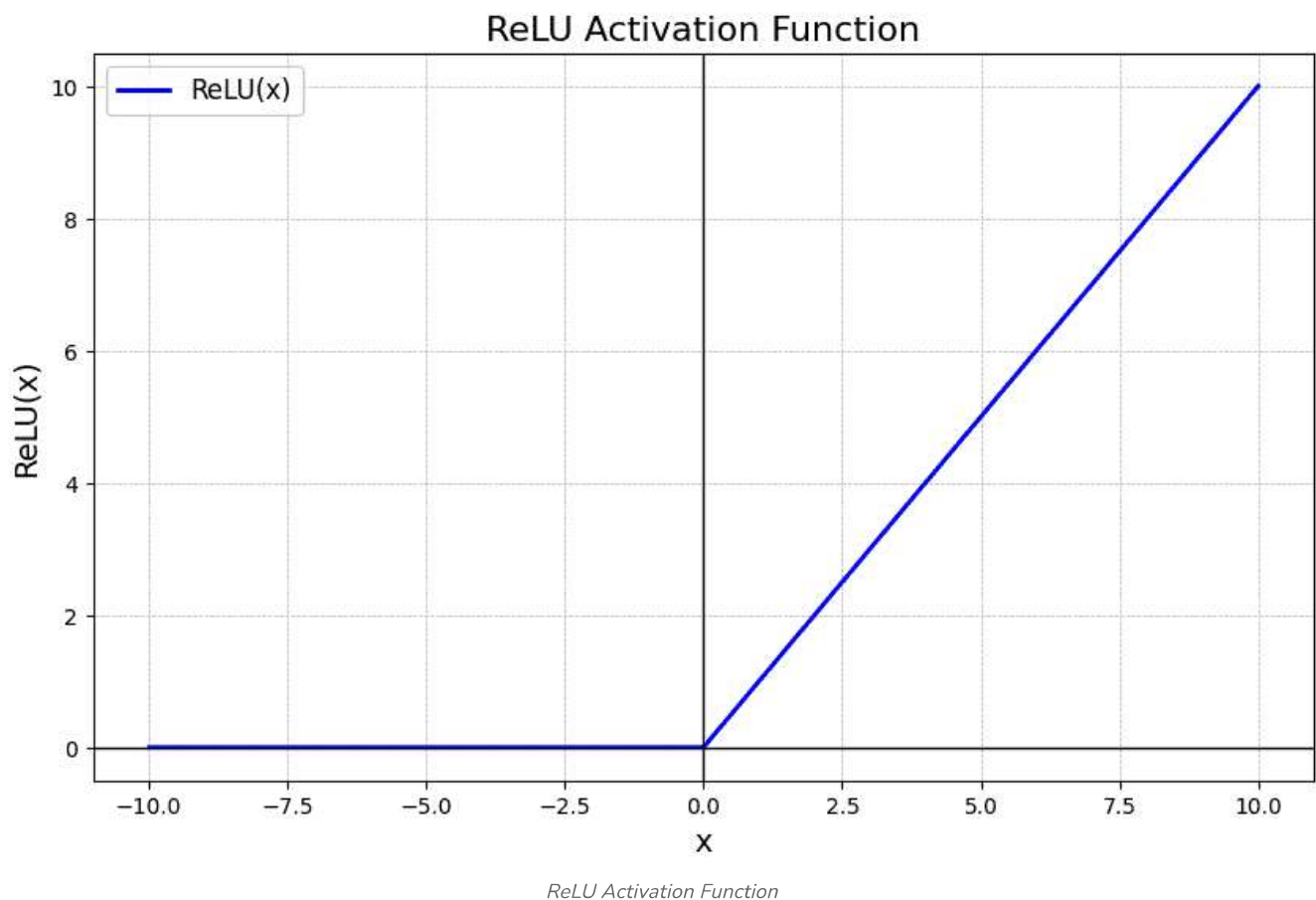


ReLU Activation Function in Deep Learning

Last Updated : 15 Apr, 2026

ReLU is a widely used activation function in deep learning that outputs the input directly if it is positive and returns zero otherwise. Its simplicity and efficiency make it a default choice in many neural network architectures, helping models learn complex patterns while reducing issues like the [vanishing gradient problem](#).

- Allows positive values to pass unchanged and sets negative values to zero.
- Simple and computationally efficient activation function.
- Helps maintain non-linearity in neural networks.
- Reduces the vanishing gradient problem compared to older functions.



Mathematical Form

The ReLU function can be described mathematically as follows:

$$f(x) = \max(0, x)$$

Where:

- x is the input to the neuron.
- The function returns x if x is greater than 0.
- If x is less than or equal to 0, the function returns 0.

The formula can also be written as:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

Why is ReLU Popular?

- **Simplicity:** ReLU is computationally efficient as it involves only a thresholding operation. This simplicity makes it easy to implement and compute, which is important when training deep neural networks with millions of parameters.
- **Non-Linearity:** Although it seems like a piecewise linear function, ReLU is still a non-linear function. This allows the model to learn more complex data patterns and model intricate relationships between features.

Q: Why did the ReLU activation function break up with its partner?

***Answer:** Because it just couldn't handle the negative energy!*

- **Sparse Activation:** ReLU's ability to output zero for negative inputs introduces sparsity in the network, meaning that only a fraction of neurons activate at any given time. This can

DSA Practice Problems C C++ Java Python JavaScript Data Science Machine Learning Courses



backpropagation, as its derivative is simple—either 0 (when the input is negative) or 1 (when the input is positive). This helps to avoid the vanishing gradient problem, which is a common issue with sigmoid or tanh activation functions.

ReLU vs. Other Activation Functions

Activation Function	Formula	Output Range	Advantages	Disadvantages	Use Case
ReLU	$f(x) = \max(0, x)$	$[0, \infty)$	• Simple and efficient • Reduces vanishing gradient	• Dying ReLU problem • Unbounded output	Hidden layers in deep networks

Activation Function	Formula	Output Range	Advantages	Disadvantages	Use Case
			Sparse activation		
Leaky ReLU	$f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$	$(-\infty, \infty)$	- Prevents dying ReLU - Allows small gradient for negative values	Requires manual tuning of α	Hidden layers (ReLU alternative)
PReLU	Same as Leaky ReLU (learns α)	$(-\infty, \infty)$	- Learns optimal slope - Better performance than ReLU in some cases	- Risk of overfitting - More parameters	Deep networks where ReLU fails
Sigmoid	$f(x) = \frac{1}{1+e^{-x}}$	$(0, 1)$	- Smooth output - Good for probabilities	- Vanishing gradient - Not zero-centered	Output layer (binary classification)
Tanh	$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$(-1, 1)$	- Zero-centered output - Better than sigmoid	Vanishing gradient	Hidden layers (normalized data)
ELU	$f(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$	$(-\alpha, \infty)$	- Smooth negative values - Faster convergence	Slower computation than ReLU	Deep networks
Softmax	$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$	$(0, 1)$ (for each)	Outputs probabilities	Computationally expensive	Output layer (multiclass)

Activation Function	Formula	Output Range	Advantages	Disadvantages	Use Case
	$\sum e^{x_j}$	class)	Handles multi-class tasks	Can cause vanishing gradients	classification

Drawbacks of ReLU

While ReLU has many advantages, it also comes with its own set of challenges:

- **Dying ReLU Problem:** One of the most significant drawbacks of ReLU is the "dying ReLU" problem, where neurons can sometimes become inactive and only output 0. This happens when large negative inputs result in zero gradient, leading to neurons that never activate and cannot learn further.
- **Unbounded Output:** Unlike other activation functions like sigmoid or tanh, the ReLU activation is unbounded on the positive side, which can sometimes result in exploding gradients when training deep networks.
- **Noisy Gradients:** The gradient of ReLU can be unstable during training, especially when weights are not properly initialized. In some cases, this can slow down learning or lead to poor performance.

Variants of ReLU

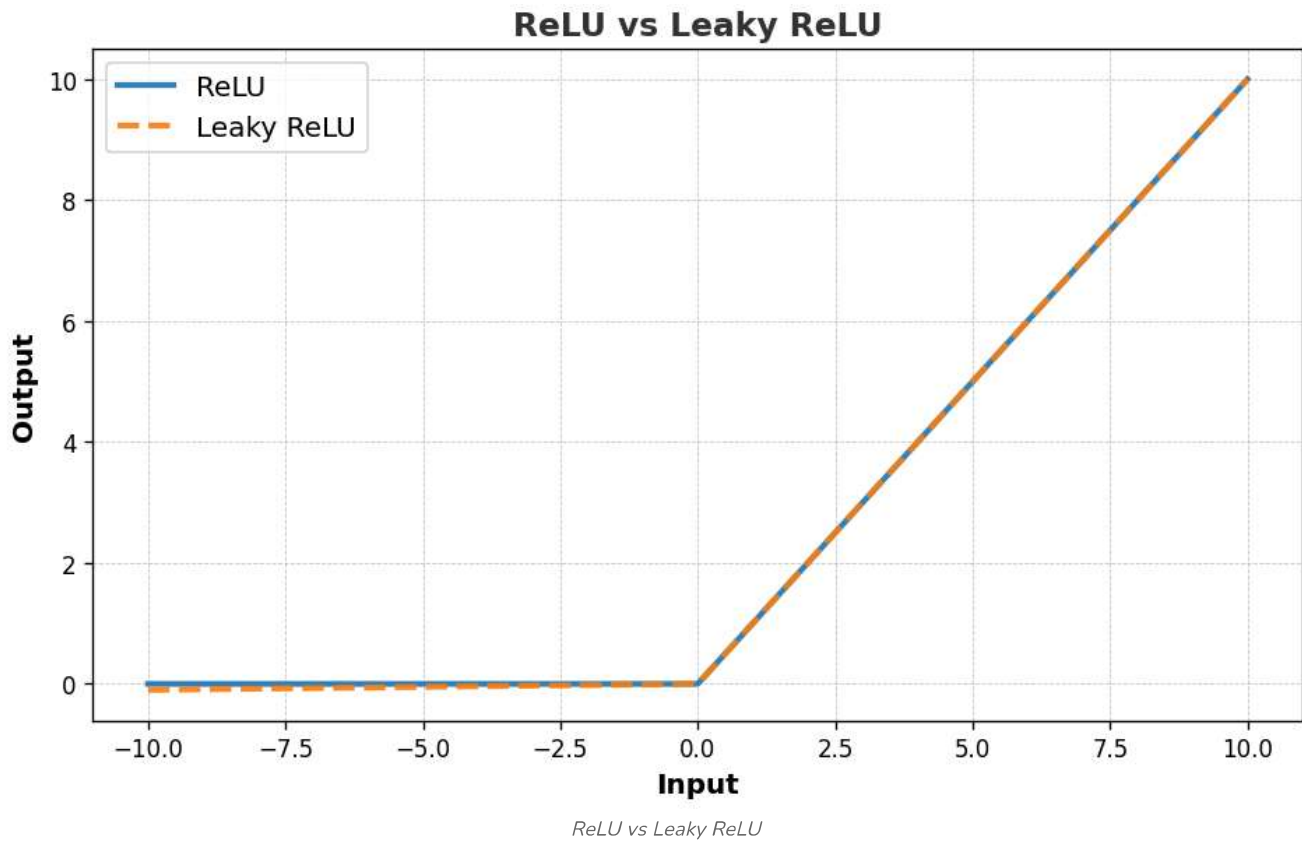
To mitigate some of the problems associated with the ReLU function, several variants have been introduced:

1. Leaky ReLU

Leaky ReLU introduces a small slope for negative values instead of outputting zero, which helps keep neurons from "dying."

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$

where α is a small constant (often set to 0.01).



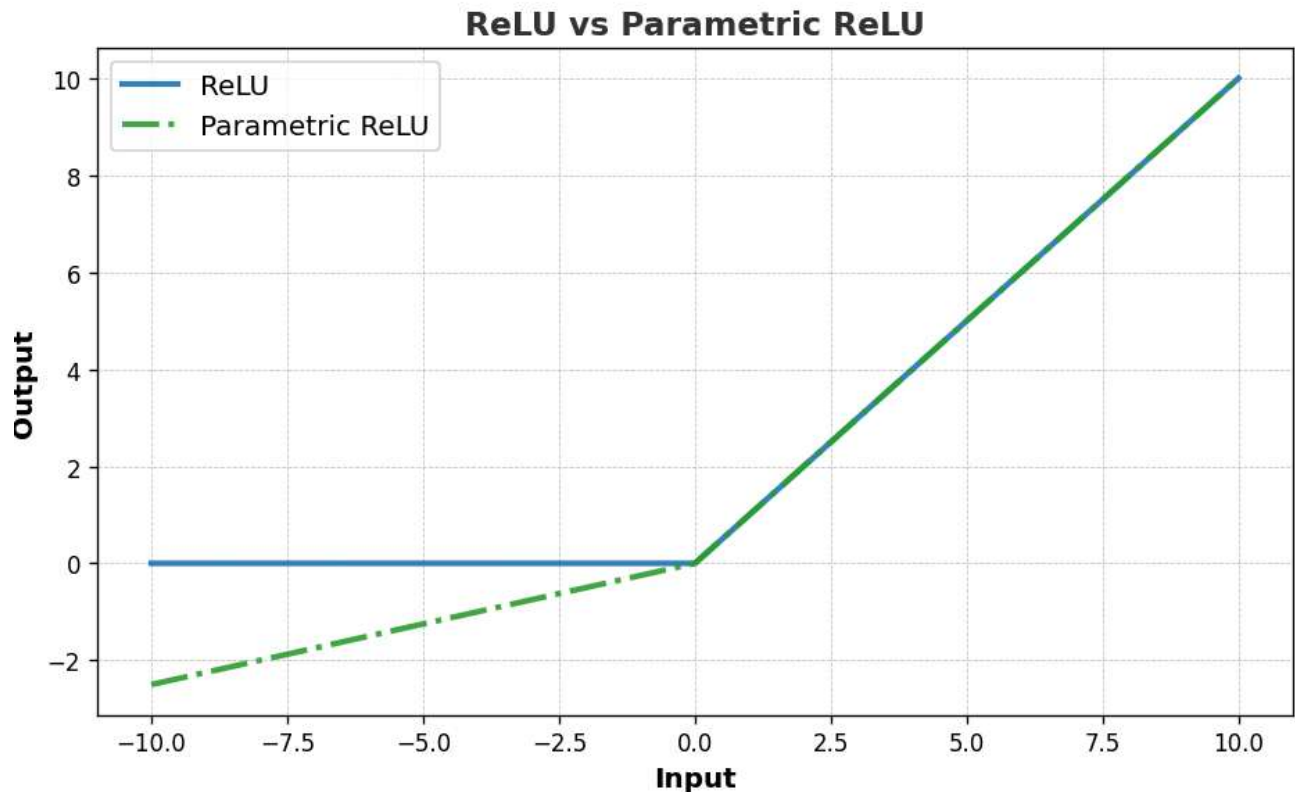
2. Parametric ReLU

Parametric ReLU (PReLU) is an extension of Leaky ReLU, where the slope of the negative part is learned during training. The formula is as follows:

$$\text{PReLU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha \cdot x & \text{if } x < 0 \end{cases}$$

Where:

- x is the input.
- α is the learned parameter that controls the slope for negative inputs. Unlike Leaky ReLU, where α is a fixed value (e.g., 0.01), PReLU learns the value of α during training.



In PReLU, α can adapt to different training conditions, making it more flexible compared to Leaky ReLU, where the slope is predefined. This allows the model to learn the best negative slope for each neuron during the training process.

3. Exponential Linear Unit (ELU)

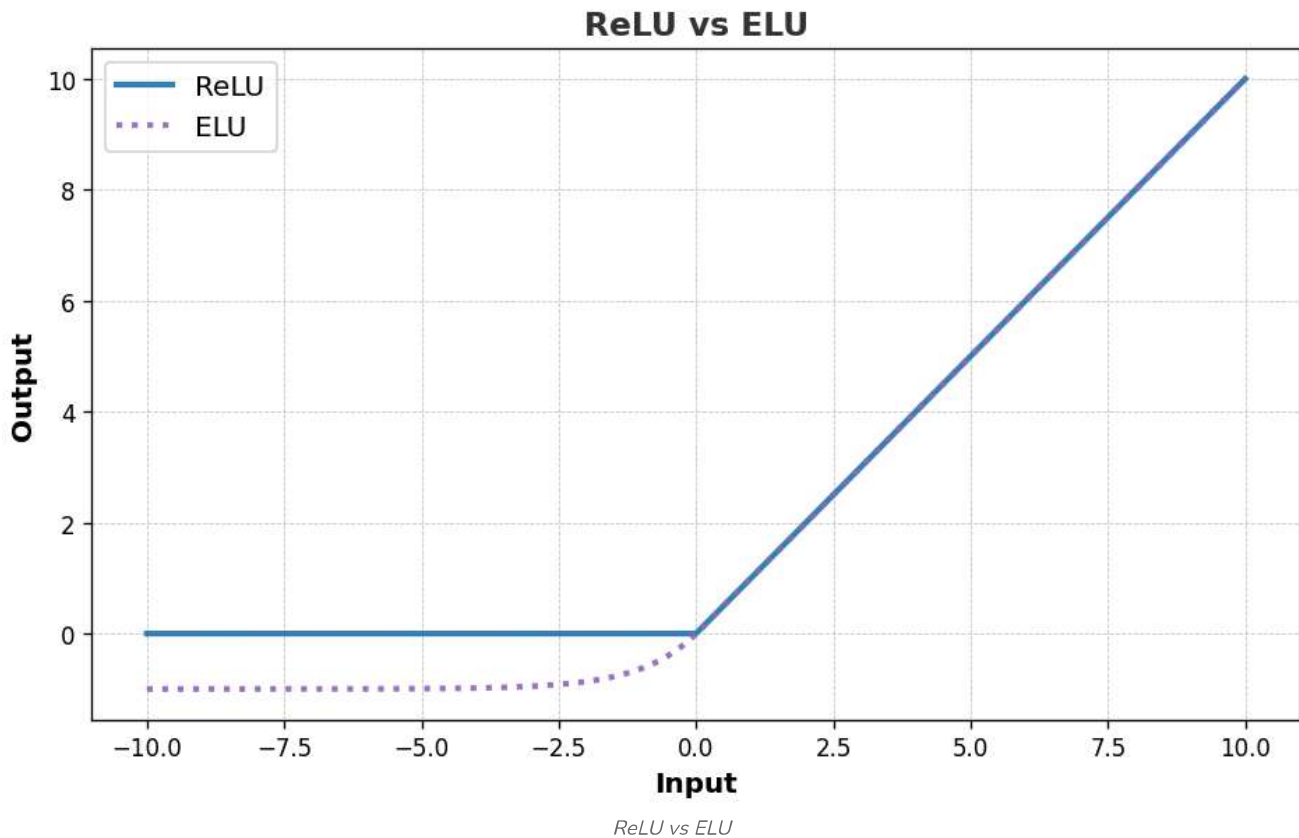
Exponential Linear Unit (ELU) adds smoothness by introducing a non-zero slope for negative values, which reduces the bias shift. It's known for faster convergence in some models.

The formula for Exponential Linear Unit (ELU) is:

$$\text{ELU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha(\exp(x) - 1) & \text{if } x < 0 \end{cases}$$

Where:

- x is the input.
- α is a positive constant that defines the value for negative inputs (often set to 1).
- For $x \geq 0$, the output is simply x (same as ReLU).
- For $x < 0$, the output is an exponential function of x , shifted by 1 and scaled by α .



When to Use ReLU?

- **Handling Sparse Data:** ReLU helps with sparse data by zeroing out negative values, promoting sparsity and reducing overfitting.
- **Faster Convergence:** ReLU accelerates training by preventing saturation for positive inputs, enhancing gradient flow in deep networks.

But, in cases where your model suffers from the *"dying ReLU"* problem or unstable gradients, trying alternative functions like Leaky ReLU, PReLU, or ELU could yield better results.

ReLU Activation in PyTorch

The following code defines a simple neural network in [PyTorch](#) with two fully connected layers, applying the ReLU activation function between them, and processes a batch of 32 input samples with 784 features, returning an output of shape [32, 10].

```
import torch
import torch.nn as nn

class SimpleNeuralNetwork(nn.Module):
    def __init__(self):
        super(SimpleNeuralNetwork, self).__init__()
        self.fc1 = nn.Linear(784, 128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

```
model = SimpleNeuralNetwork()

input_tensor = torch.randn(32, 784)

output = model(input_tensor)
print(output.shape)
```

Output:

```
torch.Size([32, 10])
```

The ReLU activation function has revolutionized deep learning models, helping networks converge faster and perform better in practice. While it has some limitations, its simplicity, sparsity, and ability to handle the vanishing gradient problem make it a powerful tool for building efficient neural networks. Understanding ReLU's strengths and limitations, as well as its variants, will help you design better deep learning models tailored to your specific needs.

Comment

A alka19...

8

Article Tags:[Deep Learning](#)[AI-ML-DS](#)[AI-ML-DS With Python](#)

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company

About Us
Legal
Privacy
Policy
Careers
Contact Us
Corporate
Solution
Campus
Training

Explore

POTD
Practice Problems
Connect
Blogs
Upskill
Courses

Tutorials

Programming Languages
DSA
Web
Technology
AI, ML &
Data Science
DevOps
CS Core
Subjects
GATE
School
Subjects
Software and Tools

Courses

ML and Data Science
DSA and Placements
Web Development
Data Science
Programming Languages
DevOps &
Cloud
GATE
MongoDB
Certifications

Preparation Corner

Interview Corner
Aptitude
Puzzles
GfG 160
System Design



@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved